

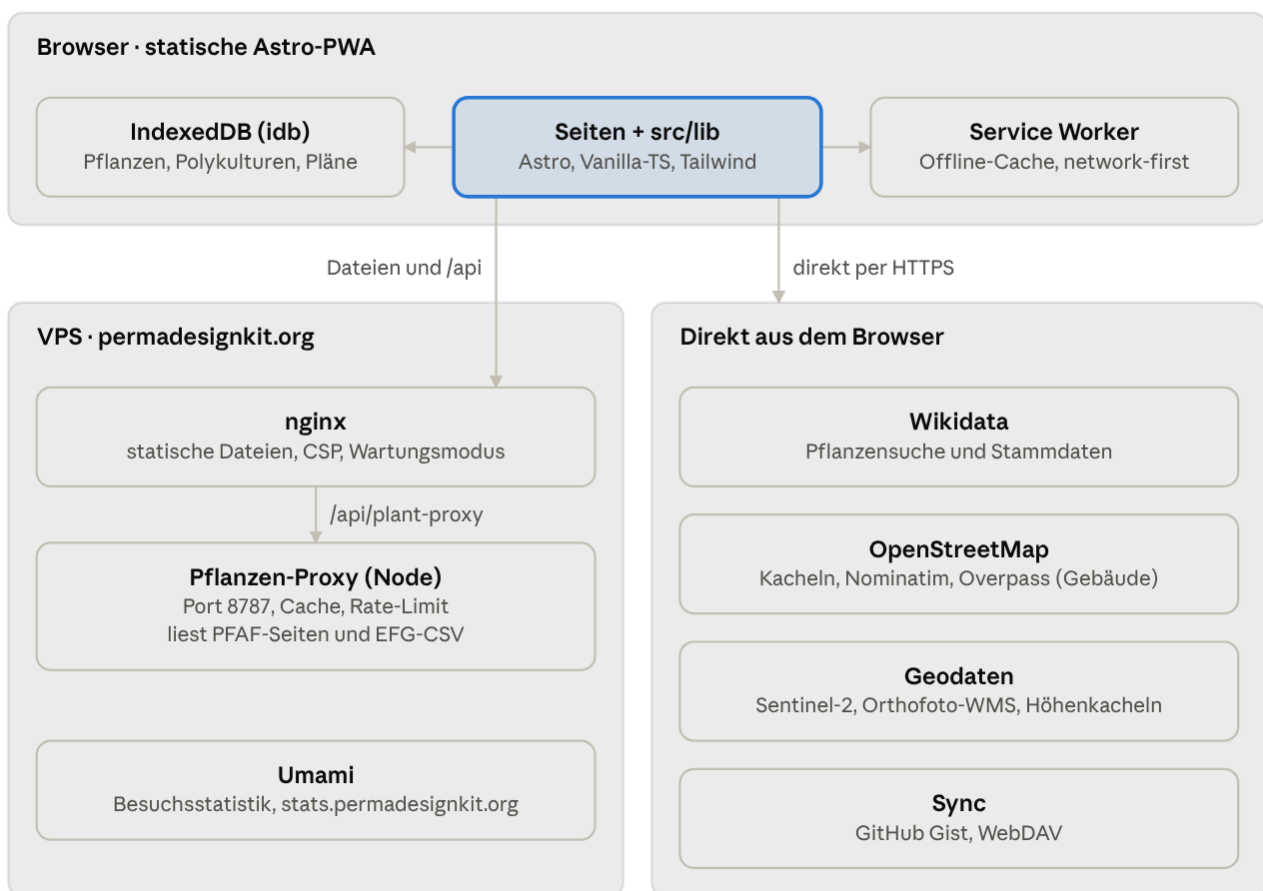
Perma Design Kit – Architektur & Werkzeuge

Oct 4, 2026 · @Andi

Das Perma Design Kit ist eine statische Astro-PWA ohne Backend für Nutzerdaten: alles liegt in der IndexedDB des Browsers, einziger Servercode ist ein kleiner Node-Proxy für PFAF- und EFG-Pflanzendaten.

Auf einen Blick

Nutzerdaten bleiben im Browser – der Server liefert nur Dateien und PFAF-Daten



Systemarchitektur · Browser, VPS, externe Dienste

Der Browser holt Seiten und `/api/plant-proxy` vom eigenen VPS; alle anderen Datenquellen fragt er direkt an, die Nutzerdaten verlassen das Gerät nur beim Sync oder Teilen.

Frontend

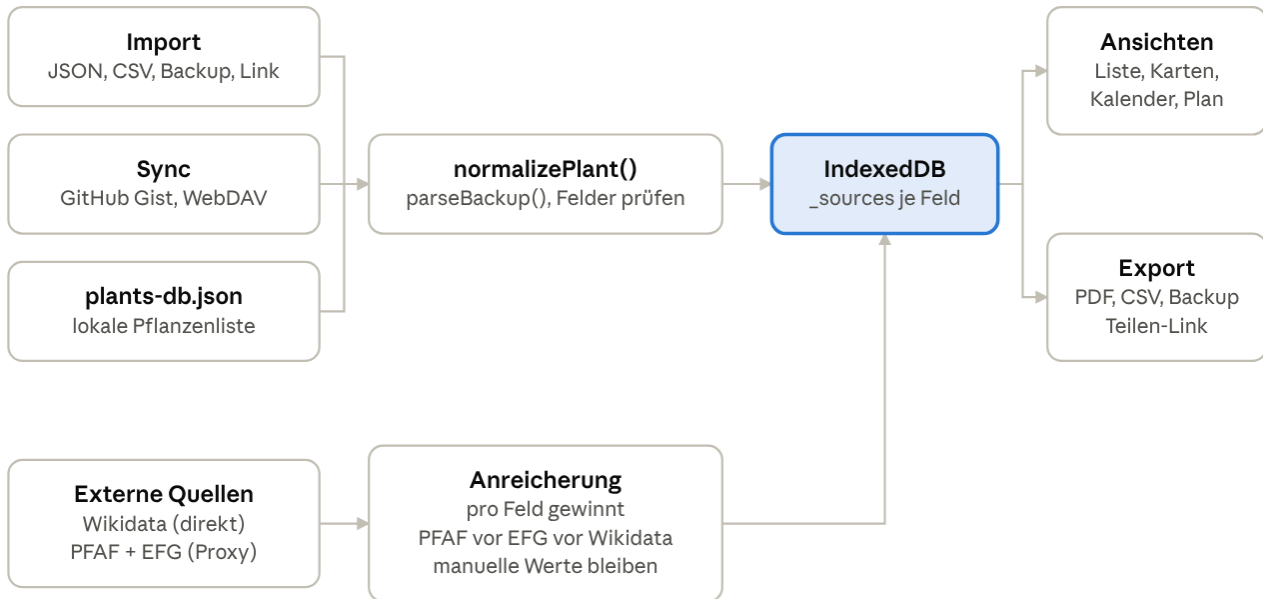
Das Frontend ist Astro 7 im statischen Modus ohne UI-Framework: jede Seite bringt ihre Interaktivität als Vanilla-TypeScript mit, gemeinsame Logik liegt in rund 45 Modulen unter `src/lib/`.

Bereich	Umsetzung
Framework	Astro 7, <code>output: 'static'</code> , keine Komponentenbibliothek, kein Komponentenordner
Interaktivität	Vanilla-TS in <code><script></code> -Blöcken je Seite, geteilte Logik in <code>src/lib/</code>
Styling	Tailwind CSS 4 über <code>@tailwindcss/vite</code> , <code>global.css</code> , Dark Mode
Layout	eine gemeinsame <code>Layout.astro</code>
Mehrsprachigkeit	eigenes DE/EN-System: <code>i18n/core.ts</code> + ein <code>dict-*.ts</code> pro Seite, <code>data-i18n</code> -Attribute
PWA	<code>manifest.json</code> ; <code>sw.js</code> network-first mit Offline-Cache, <code>/api/</code> ausgenommen; Version per <code>version-sw.mjs</code>
Pre-Paint	<code>public/boot.js</code> (Theme, Sprache) – die CSP verbietet Inline-Skripte

Seiten: `index` (Pflanzen in Raster, Liste, Karten), `kalender`, `polykulturen`, `gartenplan`, `settings`, `teilen` sowie `hilfe`, `impressum`, `datenschutz`; `cards` leitet nur auf `/?view=cards` um.

Datenfluss

Alles läuft durch die Normalisierung, bevor es in die IndexedDB kommt



Datenfluss · von Eingang und Anreicherung bis Ausgabe

Jeder Eingang läuft durch `normalizePlant()` bzw. `parseBackup()`; die Anreicherung schreibt nur Felder, deren bisherige Quelle niedriger rangiert, und vermerkt die Herkunft in `_sources`.

Fachliche Module

Drei Fremdbibliotheken tragen die schweren Teile: pdf-lib für den Export, Leaflet für Karten und three.js für die 3D-Ansicht; der Rest ist eigener Code.

Thema	Module in <code>src/lib/</code>	Bibliotheken und Dienste
Pflanzenliste, Filter, Detail	<code>plant-list</code> , <code>plant-detail</code> , <code>plant-search</code> , <code>plant-fields</code>	-
Karten-Rendering (Poly, Streifen, Baumscheibe)	<code>card-html</code> , <code>card-canvas</code> , <code>baumscheibe-render</code> , <code>baumscheibe-mapping</code>	SVG-Vorlagen mit Inkscape-Labels
PDF-Export	<code>pdf-export</code> , <code>table-pdf</code>	pdf-lib (roher <code>flateStream</code> statt <code>embedPng</code> ; Baumscheibe als JPEG bzw. Firefox-Druckfenster)

Thema	Module in <code>src/lib/</code>	Bibliotheken und Dienste
QR-Codes	<code>share</code>	<code>qrcode</code>
Polykulturen	<code>polyculture-layout</code> , <code>circle-pack</code> , <code>function-coverage</code>	-
Gartenplan 2D	<code>gartenplan-render</code> , <code>-geometry</code> , <code>-extent</code> , <code>plan-warnings</code>	eigenes SVG, Zoom/Pan per <code>viewBox</code>
Karten und Geo	<code>gartenplan-geo</code> , <code>-background</code> , <code>-mapdraw</code> , <code>-locate</code> , <code>-leaflet</code> , <code>ortho-sources</code>	Leaflet (per <code>import()</code>), OSM-Kacheln, Sentinel-2 (EOX), Nominatim, Orthofoto-WMS der Länder sowie AT/CH/NL
3D, Sonne, Gelände	<code>gartenplan-3d</code> , <code>plant-mesh-3d</code> , <code>growth-model</code> , <code>sun-position</code> , <code>sun-hours</code> , <code>terrain</code> , <code>osm-buildings</code>	<code>three.js</code> , Terrarium-Höhenkacheln (AWS), Overpass-API
Sync und Teilen	<code>sync</code> , <code>share</code>	GitHub Gist, WebDAV, URL-Fragment <code>#d=</code>

Server

Auf dem VPS laufen nur nginx, der Pflanzen-Proxy und Umami; Nutzerdaten erreichen den Server nie.

Komponente	Aufgabe	Details
<code>plant-proxy-server.mjs</code>	PFAF-Scraping und EFG-Daten für den Browser (CORS)	reiner Node, systemd-Unit <code>plant-proxy</code> , Port 8787; Cache 24 h Treffer / 10 min Fehlschläge; 15 s Upstream-Timeout; Rate-Limit pro <code>X-Real-IP</code>
<code>pfaf-parse.mjs</code>	PFAF-HTML in <code>PlantData</code> -Felder	rein, mit HTML-Fixtures getestet
<code>efg.mjs</code>	Edible-Forest-Gardens-Tabelle dekodieren	Daten in <code>server/data/efg-species.csv</code> , Update per <code>npm run efg:update</code>

Komponente	Aufgabe	Details
nginx	statische Auslieferung + Reverse-Proxy	Webroot /var/www/permadesignkit.org , /api/plant-proxy → 127.0.0.1:8787 , strenge CSP, Wartungsmodus über /var/www/maintenance.on
Umami	Besuchsstatistik	selbst betrieben unter stats.permadesignkit.org , Do-Not-Track respektiert

NaturaDB-Scraping steckt noch im Proxy, ist aber hart abgeschaltet (Lizenzfragen, siehe ROADMAP.md). Im Dev-Betrieb leitet der Vite-Proxy /api/plant-proxy an den lokalen Prozess weiter.

Tooling und Deploy

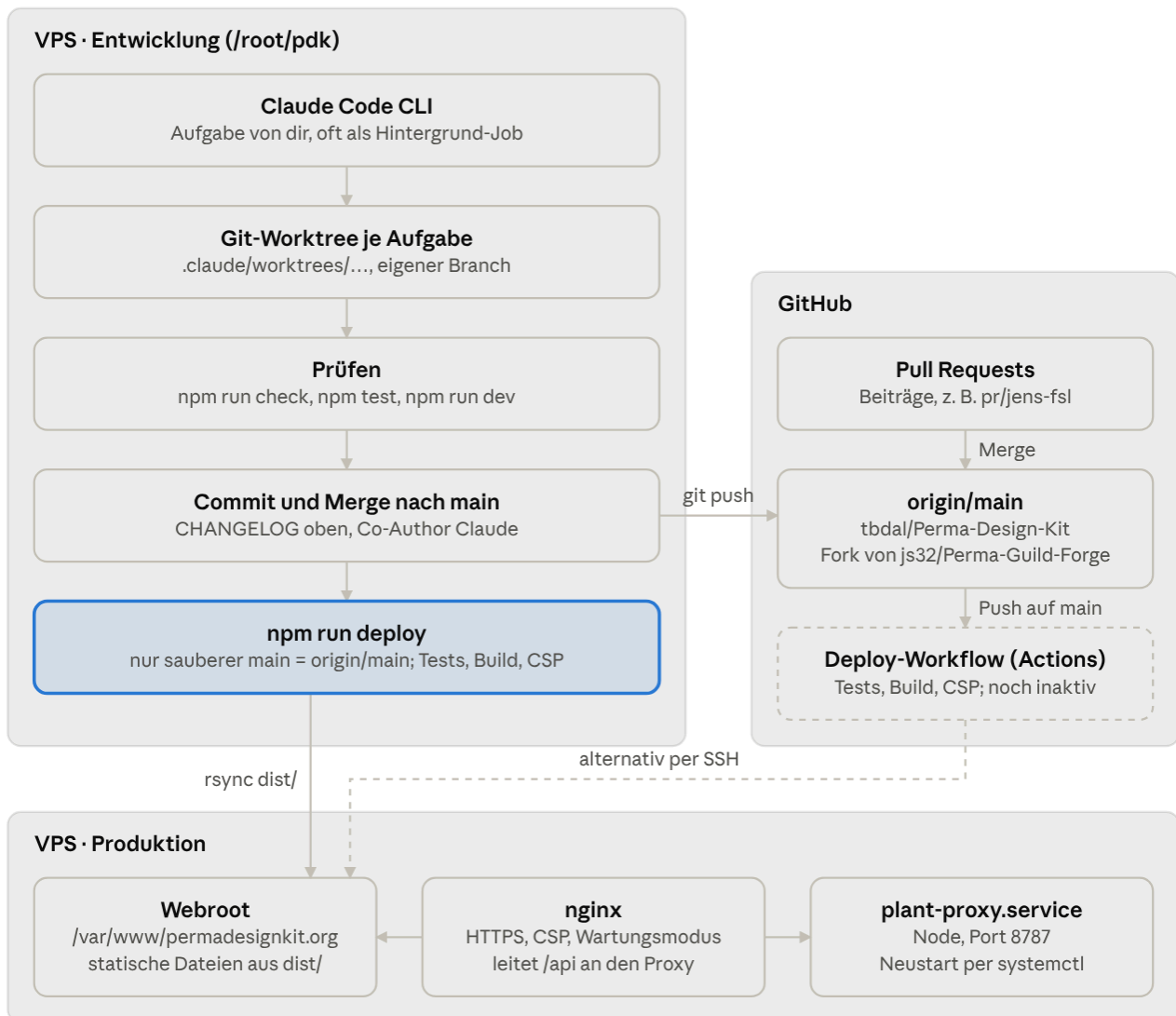
Es gibt kein CI: Tests laufen lokal vor jedem Commit und noch einmal im Deploy-Skript, das nur einen sauberen main ausliefert.

Werkzeug	Zweck	Befehl
Node ≥ 22 (nvm, .nvmrc)	Laufzeit für Build, Tests und Proxy	nvm use
TypeScript 6 + @astrojs/check	Typprüfung (der Build prüft keine Typen)	npm run check
Vitest 5	27 Testdateien in test/ , u. a. Feldtabelle, CSV, Normalisierung, Teilen, Geo, Sonnenstunden, PFAF-Parser, a11y	npm test
Astro/Vite	statischer Build nach dist/ , danach Service-Worker-Version	npm run build
check-csp.mjs	findet Inline-Skripte und on*= -Handler, die die CSP blocken würde	im Deploy
deploy.sh	Prüfungen, Tests, Build, CSP-Check, rsync nach /var/www	npm run deploy

Entwicklungsablauf

Entwickelt und ausgeliefert wird auf demselben VPS: Claude Code arbeitet in `/root/pdk`, GitHub ist die Quelle der Wahrheit, und live geht nur, was auf `origin/main` liegt.

Jede Änderung geht über main auf GitHub, bevor sie live geht



Entwicklungsablauf · VPS, Claude Code, GitHub, Webserver

1. Du gibst Claude Code im Terminal oder über `claude.ai` eine Aufgabe; längere laufen als Hintergrund-Job.
2. Jede Aufgabe bekommt einen eigenen Git-Worktree unter `.claude/worktrees/` mit eigenem Branch, damit parallele Jobs sich nicht stören.
3. Vor dem Commit laufen `npm run check` und `npm test`; zum Ausprobieren `npm run dev` mit dem lokalen Proxy.

4. Commit, Merge nach `main` und `git push` nach `tbdal/Perma-Design-Kit`; Beiträge anderer kommen als Pull Request dazu.
5. `npm run deploy` prüft, dass `main` sauber und gleich `origin/main` ist, testet, baut, prüft die CSP und kopiert `dist/` per `rsync` nach `/var/www/permadesignkit.org`.
6. `nginx` liefert sofort die neue Version aus; bei Änderungen am Proxy folgt `systemctl restart plant-proxy`, danach eine kurze Live-Prüfung per `curl`.

Offen: Die GitHub Action (CI) liegt fertig auf dem lokalen Branch `ci-action-pending`. Sie kann erst gepusht werden, wenn das Token den Scope `workflow` hat.

Alternative: Deploy über GitHub Actions

Mit dem Workflow `.github/workflows/deploy.yml` ersetzt der Push auf `main` die Schritte 5 und 6: GitHub baut, prüft und deployt, `npm run deploy` auf dem VPS entfällt.

1. Schritte 1 bis 3 bleiben gleich: Aufgabe an Claude Code, eigener Worktree, lokal testen mit `npm run dev`.
2. Commit, Merge nach `main`, `git push`. Der Push löst den Workflow `Deploy` aus; manuell geht es über Actions → Deploy → Run workflow.
3. Auf einem GitHub-Runner laufen `npm ci`, `npm run check`, `npm test`, `npm run build` und der CSP-Check, also dieselben Prüfungen wie in `deploy.sh`. Schlägt eine fehl, bleibt die alte Version live.
4. `rsync` per SSH kopiert `dist/` nach `/var/www/permadesignkit.org`, Besitzer `www-data`. Nach 1 bis 2 Minuten ist die neue Version live; der Verlauf steht unter Actions.
5. Der Proxy bleibt Handarbeit: in `/root/pdk` `git pull`, dann `systemctl restart plant-proxy`.

Worktrees, ungepushte Änderungen und ein laufendes `npm run dev` berührt der Workflow nicht. Beide Wege nicht mischen: Ein manuelles `npm run deploy` wird beim nächsten Push überschrieben. Voraussetzung sind die Repository-Secrets `VPS_SSH_KEY`, `VPS_HOST` und `VPS_KNOWN_HOSTS` (Anleitung in `pwa/SETUP.md`); der Workflow liegt noch auf dem Branch `claude/lucid-wright-ybl70q`, nicht auf `main`.

Weitere Ordner

Ordner / Datei	Inhalt
<code>legacy/</code>	ursprüngliches PowerShell-Modul <code>PermacultureTreeGuildsDesigner</code> mit Import- und Konvertierungsskripten; dessen CSV-Format importiert die PWA weiterhin

Ordner / Datei	Inhalt
psd2svg/	Python-Skripte, die aus baumscheibe.psd die SVG-Vorlage mit Inkscape-Labels erzeugen
README.md , ROADMAP.md , CHANGELOG.md , help.md	Projektdoku; der Changelog führt neueste Einträge oben
CLAUDE.md	Architektur- und Arbeitsnotizen, Quelle dieser Übersicht